

Beyond Full Fine-tuning: Finding the Limits of GPT-2 Efficient Adaptation

Stanford CS224N Default Project. (Mentor: Luke Bailey)

Alexander Zhong
Computer Science
Stanford University
ahzhong@stanford.edu

Songyu Han
Institute for Computational
and Mathematical Engineering
Stanford University
songyuh@stanford.edu

Kaitlyn Kwan
Computer Science
Stanford University
kaitkwan@stanford.edu

Abstract

Pre-trained language models such as GPT-2 have enabled transfer learning across a wide range of NLP tasks, but it remains unclear how different adaptation strategies behave across tasks with very different objectives. In this project, we study how a decoder-only Transformer can be adapted to sentiment classification, paraphrase detection, and sonnet generation using various adaptation methods, such as Bottleneck Adapters, LoRA, IA3 and Prefix Tuning. We then investigate the efficacy of hybrid PEFT, data augmentation, and Direct Preference Optimization (DPO). Our experiments show that the success of LM adapters depends strongly on the task’s linguistic complexity and data quantity, and no single adaptation strategy remains dominant across tasks. Additionally, our experiments demonstrate that PEFTs function as effective regularizers, and hybrid strategies can bridge the performance gap between PEFT and full FT in ample data regimes.

1 Introduction

The Transformer architecture marked a major step forward in NLP, demonstrating that large-scale unsupervised pretraining could yield strong, transferable representations without task-specific labeled data. Pretrained language models have since become the foundation of modern NLP, supporting tasks from text generation to classification and summarization. However, effective and efficient adaptation to downstream settings remains a central challenge.

A key difficulty is balancing adaptation quality against computational cost. Full fine-tuning often provides strong performance, but it requires updating all model parameters. Parameter-efficient fine-tuning (PEFT) methods offer an attractive alternative as they only update a small fraction of learnable parameters, but their effectiveness often varies across tasks. In this project, we closely study this tradeoff across both discriminative and generative tasks, in both data-scarce and data-rich regimes. Specifically, we compare full fine-tuning against several PEFT methods, including bottleneck adapters, LoRA, IA3, Prefix Tuning, and hybrid methods, on three downstream tasks: sentiment classification, paraphrase detection, and sonnet generation. We also explore the interaction between GPT-2 and synthetic data augmentation / direct preference optimization (DPO) on generative tasks. Overall, our results show that no single adaptation strategy dominates across all tasks: full fine-tuning is the most reliable reference baseline, especially in data-rich settings, while carefully tuned PEFT methods are highly competitive and can outperform full fine-tuning in data-scarce or more structure-sensitive settings.

2 Related Work

The Transformer architecture introduced by Vaswani et al. [1] replaced recurrent sequence models with self-attention, enabling more effective modeling of long-range dependencies and more parallelizable training. This architecture became the foundation for modern large language models (LLMs), including Google’s BERT (a bi-directional encoder) [2] and OpenAI’s GPT (an autoregressive decoder) [3].

Pre-training on large, diverse corpora to capture robust language representations has become a cornerstone of modern NLP [4]. While updating all parameters of a pre-trained model was once

standard practice, the growing size of modern LLMs makes this approach computationally demanding and creates significant deployment challenges [5]. In order to address these challenges, researchers have proposed many parameter efficient fine-tuning (PEFT) techniques, which aim to update only a small subset of model parameters while achieving comparable adaptation performance to full fine-tuning. In addition, recent works have shown that PEFT can be more effective than in-context learning in low-resource settings [6].

Sentiment analysis is a longstanding benchmark for natural language understanding [7] and a key component of modern benchmarks like GLUE and SuperGLUE [8]. Furthermore, recent research on Reinforcement Learning from Human Feedback and Direct Preference Optimization explores improving model behaviors using preference signals, which is relevant for generation tasks where generation quality cannot be pinned down with supervised fine-tuning alone. [9]

3 Approach / Methods

3.1 The GPT-2 Model Architecture

We implemented a custom GPT-2 decoder with token and positional embeddings, causal multi-head self-attention, and stacked Transformer blocks. Input tokens are mapped to learned embeddings combined with positional embeddings, then passed through transformer layers applying causal self-attention and a feedforward network with residual connections and layer normalization. The final token’s hidden representation is passed to a classification head or decoded into an output token. [10]

We have restricted our studies to GPT-2 base and GPT-2 medium models, which have pre-trained weights and tokenizers available through HuggingFace. A portion of our code was kindly provided by the CS224N course staff, but significant modifications were made to facilitate experiments.

3.1.1 Overview of adaptation tasks

Sentiment classification is formulated as a sequence classification problem. Each input sequence is encoded by GPT-2, and the final token’s hidden representation is passed to a linear prediction head.

Paraphrase detection is formulated as a cloze-style generation problem. We encode each question pair into a single prompt, and the LM is trained to predict the token yes or no.

For **Sonnet generation**, our LM is tuned to perform next-token prediction on a corpus of Shakespearean sonnets. We evaluate performance by providing the first three lines of a sonnet and allowing the LM to generate the remainder token by token.

3.2 Parameter-Efficient Fine-Tuning

We chose the following PEFT methods for this project, which we implemented ourselves to maximize compatibility with our custom GPT-2 implementation.

3.2.1 Bottleneck Adapters

Adapters, proposed in 2019 by Houlsby et al. [11], are the first successful alternative to full fine-tuning. This method injects 2-layer feed-forward networks with residual connections after the self-attention and feed-forward sub-modules in each transformer block. The dimensions of intermediate activations in the injected networks are kept small. (typically 16-64)

3.2.2 Prefix Tuning

Prefix Tuning [12] prepends trainable continuous prefixes to the key and value tensors in self-attention. Because these prefixes act like task-specific prompts in representation space, the method naturally applies to natural language generation (NLG) tasks. Mathematically, given an embedding dimension of d , sequence length of s , and a prefix length of p , the modified tensors in Prefix Tuning are:

$$K' = \text{concat}[K_\theta; K] \in \mathbb{R}^{(s+p) \times d}, \quad V' = \text{concat}[V_\theta; V] \in \mathbb{R}^{(s+p) \times d}$$

Where K_θ and V_θ are trainable prefixes of shape $p \times d$. Prefix tuning does not modify the Query.

3.2.3 Low Rank Adaptation

Low rank adaptation (LoRA) serves as a solution to sequential adapters’ inference latency and prefix tuning’s optimization challenges. [5] It adds a rank- r constraint on task-specific modifications to

dense matrix multiplication layers:

$$\Delta W = BA \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k} \text{ and } r \ll \min(d, k)$$

Equivalently, the adapted dense layers follow the following formula, and any of the 4 matrix multiplications in self-attention (Key, Query, Value, and Output Projection) can be adapted by LoRA.

$$Hx = W_0x + BAx$$

3.2.4 (IA)³

(IA)³ [6] is a lightweight alternative to LoRA that rescales intermediate activations with learned vectors in self-attention and feed-forward submodules. It changes the self-attention formula with broadcasted element-wise multiplication:

$$\text{softmax}\left(\frac{Q(l_k \odot K^\top)}{\sqrt{d_k}}\right)(l_v \odot V).$$

3.2.5 Hybrid Methods

Research has also explored interactions between PEFT modules. Hu et al. proposed that LoRA can be naturally combined with Prefix Tuning to achieve **orthogonal improvement**, but their interaction is task-dependent and often unpredictable. [5] Mao et al. proposed UniPELT to address the performance degradation of naively combining PEFTs. A learnable gating mechanism controls the influence of Prefix Tuning, Adapters, and LoRA independently based on input embeddings, and UniPELT generally outperforms individual PEFTs when task-specific data is limited.[13]

3.3 Direct Preference Optimization

After supervised fine-tuning, a language model’s generation outputs are still ill-aligned with human preference, so the behavior of a language model is further modified by Reinforcement Learning from Human Feedback (RLHF) before deployment. Rafailov et al. proposed Direct Preference Optimization [9] as a simpler alternative to RLHF, and it uses the following loss function for each input x , preferred output y_w and rejected output y_l :

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right)$$

4 Datasets

For sentiment classification, we used a smaller IMDB review dataset ($N_{\text{train}} = 1707, N_{\text{dev}} = 245, N_{\text{test}} = 489$) for binary classification as well as a larger Stanford Sentiment Treebank (SST-5) dataset ($N_{\text{train}} = 8544, N_{\text{dev}} = 1101, N_{\text{test}} = 2211$) for multinomial classification.

For paraphrase detection, we used the Quora Question Pairs (QQP) benchmark dataset. ($N_{\text{train}} = 283011, N_{\text{dev}} = 40430, N_{\text{test}} = 80860$)

Finally, for sonnet generation, we used both William Shakespeare’s original sonnets ($N_{\text{train}} = 131, N_{\text{dev}} = 12, N_{\text{test}} = 12$) as well as an LLM-generated synthetic dataset with 200 Shakespearean-style sonnets and 200 non-Shakespearean-style sonnets. Our synthetic dataset was either augmented to the original training split to address the data scarcity, or used to experiment with DPO after supervised fine-tuning.

5 Experiments

5.1 Experimental details

Our experiments feature task-specific training data that differ greatly in size and linguistic structure to study the adaptation behavior of GPT-2. Across all experiments, we use a cosine learning rate schedule with 10% linear warm-up and apply early stopping when validation loss plateaus.

Model Selection Experiments:

We begin with a hyperparameter search on SST-5 and CFIMDB to identify suitable learning rates and

regularization strengths for PEFT. These trials run for at most 25 epochs. LoRA with $r_q = r_v = 8$ is used as a representative PEFT method during this initial search.

PEFT Ablation Studies:

We then perform rigorous ablation studies for LoRA and Bottleneck Adapters. For adapters, we benchmark bottleneck dimensions $\{16, 32, 64, 128, 256\}$ using two different learning rates, and for LoRA, we sweep ranks $\{2, 4, 8\}$ and various target-module combinations. Learning rate for LoRA is scaled inversely with the number of target modules for optimizer stability.

GPT-2 Medium PEFT Comparison:

We apply the best performing PEFT candidates to the larger GPT-2 medium model. Here, we comprehensively compare Adapters, LoRA, (IA)³, Prefix Tuning and hybrids on the SST-5 dataset, and we benchmark hybrid approaches against a LoRA baseline for Quora Question Pairs (QQP).

Generation Experiments:

For sonnet generation, we study two extensions beyond standard supervised fine-tuning: synthetic data augmentation and Direct Preference Optimization (DPO). These experiments allow us to test whether additional training data or preference-based optimization can improve generation quality beyond naive full fine-tuning and PEFT approaches.

Computational Resources:

Our main computational resource is a Google Cloud VM with an Nvidia L4 GPU (24 GB), but Google Colab (with L4 or T4 GPUs) and local GPUs (AMD Radeon Pro 5600M 8GB) are also involved in experiment prototyping and debugging. Compute is determined by dataset size, with QQP being the most expensive, where each epoch takes 2 hours for full fine-tuning and one hour for PEFT. Sentiment classification and sonnet generation were less expensive to train, although sonnet evaluation suffered from the bottleneck of LM sequential decoding.

5.2 Evaluation Methods

For sentiment classification and paraphrase detection, we use classification accuracy on the validation / development split, according to the standards set forth by the General Language Understanding Evaluation Benchmark (GLUE) [8]. We also closely monitor the training-validation gap for overfitting identification.

For sonnet generation, we compare the model’s generated sonnets against the validation ground truths using **Character Level F-Score** (ChrF). ChrF is a model-free metric where a higher score indicates that the generated sonnet more closely matches the reference, suggesting stronger performance.

5.3 Results

5.3.1 Baseline Results

We use full fine-tuning of GPT-2 base as our primary reference baseline. This provides a reference point for all later adaptation methods. Hyperparameters are selected using coarse sweeps over learning rate, regularization, and, for generation, sampling parameters. The baseline results are shown in Table 1, and additional results for hyperparameter sweeps can be found in the appendix.

Task	LR	Batch Size	Epochs	Key Params	Valid Acc. / ChrF
SST-5	5e-5	256	10	Weight Decay 5e-2	0.516
CFIMDB	5e-5	32	10	Weight Decay 5e-2	0.976
QQP / Paraphrase	1e-5	4	3	N/A	0.893
Sonnet	5e-5	8	8	p = 0.9, T = 1.2	41.999

Table 1: GPT-2 Base Baselines

5.3.2 PEFT Hyperparameter Experiments for Sentiment Classification

Table 2 summarizes our initial hyperparameter search. On sentiment classification tasks, we found that LoRA is mostly insensitive to weight decay, but the interaction between learning rates and adaptation success diverges significantly between SST-5 and CFIMDB. While LoRA favors higher learning rates (2.5×10^{-3}) on SST-5, it achieves stronger accuracy on CFIMDB with lower learning rates (4×10^{-4}) and stronger regularization.

Learning Rate	SST-5			CFIMDB		
	Epoch	Train Acc	Valid Acc	Epoch	Train Acc	Valid Acc
4.0×10^{-4}	16	0.589	0.503	12	0.999	0.984
7.5×10^{-4}	11	0.615	0.507	10	0.998	0.984
1.0×10^{-3}	8	0.599	0.502	5	0.994	0.980
2.5×10^{-3}	6	0.630	0.510	1	0.978	0.963

Table 2: Training hyperparameter search on GPT-2 base (Summarized by taking the best of 3 runs.)

5.3.3 Ablation Studies on Adapters for Sentiment Classification

Dim	Trainable	SST-5			CFIMDB		
		LR Epoch	Train Acc	Valid Acc	LR Epoch	Train Acc	Valid Acc
16	612K (0.5%)	1e-3 14	0.716	0.530	1e-4 10	0.995	0.984
32	1.2M (0.95%)	1e-3 10	0.680	0.521	1e-4 8	0.994	0.980
64	2.4M (1.88%)	1e-3 7	0.725	0.524	1e-4 6	0.996	0.980
64	2.4M (1.88%)	2e-4 12	0.613	0.540	2e-5 8	0.989	0.980
128	4.7M (3.67%)	2e-4 9	0.624	0.529	2e-5 4	0.987	0.963
256	9.5M (7.07%)	2e-4 12	0.604	0.505	2e-5 6	0.994	0.976

Table 3: Bottleneck Adapter Performance on GPT-2 Base

In this study (Table 3), we vary adapter size from 16 to 256 (0.5% to 7.1% of trainable parameters) and we use two learning rates: 0.0001 and 0.00002.

Our results confirm that increasing bottleneck dimensions past 64 does not improve performance, and larger bottleneck dimensions (which correspond to more trainable parameters) favor a reduced learning rate on SST-5. On CFIMDB, our best-performing candidate has the least trainable parameters, suggesting that SST-5 benefits from moderate adapter capacity, whereas CFIMDB can be adapted effectively with low-capacity PEFTs.

5.3.4 Ablation Studies on LoRA for Sentiment Classification

Rank	Modules	SST-5		Modules	CFIMDB	
		LR	Valid Acc		LR	Valid Acc
2	QKVO	1.25e-3	0.522	QKVO	1.88e-4	0.984
	O	5e-3	0.519	QV	3.75e-4	0.976
4	QV	1.6e-3	0.529	V	5e-4	0.980
	O	3.2e-3	0.518	QV	2.5e-4	0.976
8	V	2e-3	0.526	V	2.5e-4	0.984
	O	2e-3	0.510	Q	2.5e-4	0.980

Table 4: LoRA: Selected top candidates for each rank. (Full results in appendix)

We report the best and second-best performing candidate for each rank in Table 4, after learning rate tuning. Our results indicate that the best performing candidate for rank 2 is to adapt all dense modules in self-attention, whereas the best performing candidate for rank 8 only adapts the value projection. For rank 4, adapting query and value seems the most reasonable (especially for SST-5), which is consistent with the literature.

Interestingly, our best performing candidates on SST-5 all share the same number of trainable parameters and similarly tuned learning rates, and this coincides with our adapter study findings that increasing trainable parameters in PEFT does not always yield additional performance gains.

5.3.5 PEFT Showdown on GPT-2 Medium

We then study the scaling behaviors of PEFT methods from GPT-2 base (124M) to GPT-2 medium (355M). For SST-5, we test the top-3 candidates from our ablation studies and report the best performer in Table 5, and we also include some hybrid methods that adapt self-attention with LoRA and adapt FFN with adapters.

Method	Configuration	Trainable	LR Epoch	Train Acc	Valid Acc
Full FT Baseline	N/A	355M (100%)	1e-5 13	0.765	0.537
Adapter	$d = 16$	1.63M (0.457%)	5e-4 6	0.672	0.559
Prefix Tuning	$n = 10$	26.8M (7.03%)	1.6e-3 16	0.641	0.540
(IA) ³	N/A	152K (0.043%)	2e-3 9	0.629	0.537
LoRA	$r_q = r_v = 4$	398K (0.112%)	8e-4 6	0.656	0.544
Adapter + LoRA	Adapter: $d = 16$, FFN LoRA: $r_q = r_v = 4$	1.21M (0.340%)	5e-4 5	0.662	0.552
<u>UniPELT</u>	Adapter: $d = 16$, FFN LoRA: $r_q = r_v = 4$	<u>1.23M (0.347%)</u>	<u>5e-4 5</u>	<u>0.673</u>	<u>0.556</u>

Table 5: GPT-2 Medium PEFT Comparison on SST-5 Dataset.

First, with appropriately tuned learning rate and configuration, all PEFT methods met or exceeded the baseline. Full fine-tuning suffered from significant **overfitting** despite small learning rates and strong regularization ($5e-2$ weight decay). This implies PEFT methods have strong implicit regularization effects in data-scarce regimes, forcing the model to learn robust task-dependent representations rather than memorizing noise.

Second, naively combining PEFTs degraded performance from our best adapter configuration, but UniPELT’s gating mechanism effectively regulates adapter and LoRA strength for different inputs, mitigating destructive interference and performance degradation. The performance drop may also reflect SST-5’s preference for flexible methods with more trainable parameters, as lightweight solutions like LoRA or (IA)³ performed notably worse.

Method	Configuration	Trainable	LR Epoch	Train, Valid Loss	Valid Acc
Full FT	N/A	355M (100%)	2e-5 5	0.030, 0.426	0.911
LoRA	$r_q = r_v = 8$	768K (0.221%)	5e-4 5	0.201, 0.253	0.889
UniPELT	LoRA: $r_q = r_v = 8$ Adapter: $d = 48$, FFN	3.25M (0.906%)	5e-4 4	0.177, 0.244	0.904

Table 6: GPT-2 Medium PEFT Comparison on Quora Question Pairs.

Table 6 documents our findings of PEFT scaling on the QQP benchmark, a task with lowered task difficulty and ample training data. As expected, full fine-tuning is highly effective, achieving a highly competitive validation metric despite minor overfitting. On the flip side, the implicit regularization effect of LoRA becomes a disadvantage in data-rich regimes. Our findings suggest LoRA suffered from underfitting, and the low-rank update constraint bottlenecked the LM’s ability to fully absorb our vast training materials. Finally, our UniPELT candidate strikes a balance between performance and efficiency: It retains 99% of the full FT performance while using less than 1% of trainable parameters. It also exhibits less overfitting.

5.3.6 Sonnet Generation with PEFT

Our generation experiments use GPT-2 base, weight decay of 0.05, and top k sampling of 50.

Rank	Mode	LR	Epochs / Batch	(p, T)	chrF
1	Full	5e-5	8 / 8	(0.9, 1.2)	41.9989
2	PEFT	1.5e-4	12 / 4	(0.95, 0.9)	41.9648
3	PEFT	1e-4	12 / 4	(0.95, 1.0)	41.9116
4	Full	2e-5	10 / 8	(0.9, 1.2)	41.7611
5	PEFT	2e-4	10 / 4	(0.95, 1.0)	41.6095

Table 7: Top 5 sonnet candidates across full FT and PEFT

We present our best configurations on sonnet generation in Table 7. While full fine-tuning remains a strong baseline, properly tuned PEFTs are highly competitive as 6 of our top 10 configurations were PEFTs. Again, full fine-tuning prefers lower learning rates, and PEFT requires larger learning rates and small batches to be competitive on our small task-specific dataset. The best performing PEFT candidate was a **hybrid method** using adapters ($d = 64$) and LoRA($r_q = r_v = 8$). (ChrF: 41.9648)

Among single PEFTs, Prefix tuning is the most competitive candidate with ChrF 41.3166, but naively combining LoRA and prefix tuning degraded performance to 41.0642. Naively adding adapters to the mix further dropped our performance to 40.4294.

Despite these findings, a properly tuned UniPELT method with gated adapters ($d = 16$), LoRA ($r_q = r_v = 4$) and prefix tuning ($n = 10$) achieved a highly competitive ChrF of **42.62** on GPT-2 medium. But unlike our GPT-2 base candidates, this model was trained using **cosine annealing with linear warm-up** and a baseline rate of $8e - 4$. This implies sophisticated hybrid PEFT methods are highly sensitive to optimization dynamics, and proper lr scheduling is crucial to minimize destructive interference between PEFTs with divergent mathematical foundations.

5.3.7 Sonnet Generation: Data Augmentation and Preference Optimization

Mode	LR	Epochs / Batch	(p, T)	chrF
Full	3e-5	8 / 8	(0.9, 1.2)	42.365
Full	5e-5	8 / 8	(0.9, 1.2)	41.503
PEFT	1.5e-4	12 / 8	(0.95, 0.9)	41.503

Table 8: Sonnet Generation: Data Augmentation

β	LR	Epochs / Batch	chrF
0.20	1e-6	3 / 3	41.7334
0.30	1e-6	3 / 3	41.2274
0.05	1e-6	3 / 3	41.0318
0.10	1e-6	3 / 3	41.0145
0.10	3e-6	2 / 3	40.9131

Table 9: Sonnet Generation: DPO

We first combine the original Shakespeare sonnets with a synthetic dataset of 200 sonnets imitating the Shakespearean style. According to Table 8, our best performing candidate with synthetic data was a fully fine-tuned base GPT-2 model with a slightly lower learning rate of 3×10^{-5} . This candidate outperformed our best GPT-2 base candidate by a significant margin, which implies that synthetic data has potential to improve performance on smaller models.

We also evaluated Direct Preference Optimization (DPO) for sonnet generation on a GPT-2 base model with PEFT. Our best candidate was achieved by using a small learning rate and larger $\beta = 0.2$. While DPO combined with PEFT did not outperform our best full fine-tuning baseline, it remained competitive. These results suggest that there is some merit to improving performance with DPO, but its success depends strongly on hyperparameter selection and the quality of preference data.

5.3.8 Performance on held-out test sets:

After completing our experiments and model selection process, we evaluated our best performing candidates for the QQP paraphrase equivalence and sonnet generation tasks on a held-out test set, which we do not have access to the true labels. All QQP candidates use a weight decay of 0.005, while the Sonnet candidate uses 0.05. All learning rates are scheduled using a cosine learning rate scheduler with linear warmup. Batch size is 8 across both tasks. A gating mechanism was applied to the LoRA and adapter components in UniPELT.

Task	Pre-Trained Model	Method	Fine Tuning Config	LR Epoch	Valid	Test
QQP	GPT-2 Medium	Full FT	N/A	2e-5 5	0.911	0.905
QQP	GPT-2 Medium	UniPELT	LoRA ($r_q = r_v = 8$), Adapter (FFN, $d = 48$)	5e-4 2	0.904	0.890
Sonnet	GPT-2 Medium	UniPELT	LoRA ($r_q = r_v = 4$), Adapter (FFN, $d = 16$), Prefix ($n = 10$)	8e-4 10	42.62	42.07

Table 10: Validation and Test split performance of top candidate models

6 Analysis / Discussion

6.1 Sentiment Classification

Our results highlight a clear difference in task nature between SST-5 and CFIMDB. All methods adapted well to CFIMDB, while we frequently saw underfitting or overfitting problems in SST-5. A likely reason is that SST-5 contains inherently ambiguous class boundaries, especially between neighboring labels such as *negative* and *very negative*. In addition, SST-5 samples are shorter, so correct classifications often rely on subtle semantic cues and fine-grained language understanding. In contrast, CFIMDB reviews are longer and repetitive, often containing explicit sentiment-bearing

phrases that provide strong lingual supervision. Additionally, CFIMDB samples strongly resemble the pretraining material of GPT-2, which consisted largely of webpages written in modern English. These structural differences in samples help explain why most methods performed well on CFIMDB while SST-5 is much more picky about architecture and hyperparameters.

6.2 Paraphrase Detection

We saw that framing the QQP task as a cloze-style generation problem is highly effective with GPT-2 as it can leverage autoregressive properties to generate the class labels directly. The samples in QQP is qualitatively similar to CFIMDB in linguistic structure, and we empirically observed very strong performance for both full fine-tuning and PEFT. Full fine-tuning is the canonical choice because our ample training data is sufficient to teach large LMs an exhaustive inventory of natural language representations that are useful to determine semantic equivalence, but we also saw that property tuned PEFT hybrids are extremely close to full fine-tuning performance. Although not included in our main results, we also saw that poorly tuned PEFT hybrids (eg. naive LoRA + Prefix) lead to unstable optimization and representation collapse, with some runs appearing to stall early in training.

6.3 Sonnet Generation

Sonnet generation was the most challenging task, requiring the model to learn local fluency as well as longer-range structural properties like line boundaries, cadence, and stylistic consistency. The linguistic gap between Shakespearean sonnets and GPT-2’s pretraining corpus demands significant behavioral adaptation, yet the small training dataset makes overfitting a persistent challenge.

Early full fine-tuning runs produced outputs with inconsistent line counts, weak rhyme patterns, and off vocabulary. As our experiments continued, the model first learned surface-level style before gradually capturing deeper structural properties of Shakespearean sonnets. Our data augmentation and DPO methods helped smaller models by exposing it to more Shakespearean vocabulary and repeated poetic structure, but the same tricks degraded the performance of larger models like GPT-2 large and XL (not shown in tables), likely because synthetic texts of lower quality than Shakespeare’s original work disturbed strong foundational language representations embedded in large LMs.

For PEFT, Prefix Tuning is generally preferable and a carefully tuned PEFT method can be competitive to full fine-tuning. Generation hyperparameters also greatly influenced evaluation, with poorly tuned P , K , or T producing degenerate samples. Finally, we validated the challenges of evaluating natural language generation: while ChrF is easy to operationalize, most differences between scores like 40.8 and 42.4 reflect statistical noise rather than meaningful differences in generation quality.

7 Conclusion and Future Work

We studied the adaptation behaviors GPT-2 on three downstream tasks: sentiment classification, paraphrase detection, and sonnet generation. After establishing strong full fine-tuning baselines, we rigorously evaluated a wide array of PEFT methods including bottleneck adapters, LoRA, (IA)³, Prefix Tuning, UniPELT, hybrid PEFTs, as well as synthetic data augmentation and DPO. Our best validation results were 0.559 on SST-5, 0.984 on CFIMDB, 0.911 on paraphrase detection, and 42.62 chrF on sonnet generation. On the held-out test splits, we achieved 0.905 on paraphrase detection and 42.07 chrF on sonnet generation.

Our results show that effective LM adaptation is strongly task-dependent. While full fine-tuning was the most reliable baseline, it was prone to overfitting, whereas PEFT methods, in addition to saving compute, acted as structural regularizers that exceeded full fine-tuning in data-scarce regimes. Light-weight methods like LoRA and (IA)³ sufficed for simpler tasks, while adapters and UniPELT excelled in difficult or data-rich tasks. Prefix tuning showed unique strength on generative tasks with limited training sample, and data augmentation and DPO yielded additional performance gains.

Key limitations include limited hyperparameter sweeps for DPO and hybrid PEFT methods due to compute constraints, persistent struggles with sonnet structural properties despite strong quantitative results, and limited exploration beyond GPT-2 Base and Medium, leaving larger, more capable variants of GPT-2 underexplored. Future work should scale up preference data for DPO, develop explicit structural signals for sonnet generation, and evaluate hybrid adaptation strategies on larger backbones.

Team contributions (Required for multi-person team)

Songyu Han: Songyu performed extensive literature review and completed GPT-2 and PEFT (Adapters, IA3, LoRA, and Prefix tuning) core implementations in the first project stage. Designed experiments for sentiment / paraphrase tasks, and performed adapter / LoRA ablation studies as well as PEFT transfer studies to GPT-2 Medium and Large. Helped the rest of the team on sonnet generation and final report during the final project stage.

Kaitlyn Kwan: Kaitlyn contributed to the write ups, including the proposal, milestone report, final report, and poster. For implementation, she implemented the baseline sentiment task, created the synthetic datasets for data augmentation, and implemented DPO. She also performed hyperparameter sweep experiments for full fine-tuning and PEFT for the Sonnet task, data augmentation for various dataset extensions, and DPO.

Alexander Zhong: Alex contributed to all of the writeups and helped implement the baseline sonnet generation and paraphrase detection implementation along with some of the PEFT code. He also performed hyperparameter sweeps experiments for full fine-tuning for paraphrase detection and full fine-tuning, PEFT, data augmentation for sonnet generation.

Project Logistical Information

- Mentor: Luke Bailey
- Late Days:

Team member	Late days remaining
Alexander Zhong	5
Songyu Han	5
Kaitlyn Kwan	2
Total pooled	12
Team size	3
Late days applied for the report	3

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.
- [4] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault, editors, *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8342–8360, Online, July 2020. Association for Computational Linguistics.
- [5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *CoRR*, abs/2106.09685, 2021.

- [6] Haokun Liu, Derek Tam, Mohammed Muqeeth, Jay Mohta, Tenghao Huang, Mohit Bansal, and Colin Raffel. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning, 2022.
- [7] Richard Socher et. al. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013.
- [8] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.
- [9] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.
- [10] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [11] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR, 09–15 Jun 2019.
- [12] Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation, 2021.
- [13] Yuning Mao, Lambert Mathias, Rui Hou, Amjad Almahairi, Hao Ma, Jiawei Han, Wen-tau Yih, and Madian Khabsa. Unipelt: A unified framework for parameter-efficient language model tuning. *CoRR*, abs/2110.07577, 2021.

A Appendix

Learning Rate	Weight Decay	Stanford Sentiment Treebank - 5			CFIMDB Sentiment Classification		
		Train Acc	Valid Acc	Epoch	Train Acc	Valid Acc	Epoch
4.0×10^{-4}	5.0×10^{-4}	0.589	0.500	16	0.999	0.984	12
4.0×10^{-4}	5.0×10^{-3}	0.590	0.500	16	0.999	0.984	12
4.0×10^{-4}	5.0×10^{-2}	0.589	0.503	16	0.999	0.984	12
7.5×10^{-4}	5.0×10^{-4}	0.613	0.507	11	0.979	0.980	5
7.5×10^{-4}	5.0×10^{-3}	0.615	0.507	11	0.981	0.980	5
7.5×10^{-4}	5.0×10^{-2}	0.662	0.504	16	0.998	0.984	10
1.0×10^{-3}	5.0×10^{-4}	0.599	0.502	8	0.994	0.976	5
1.0×10^{-3}	5.0×10^{-3}	0.600	0.500	8	0.994	0.980	5
1.0×10^{-3}	5.0×10^{-2}	0.599	0.500	11	0.998	0.976	7
2.5×10^{-3}	5.0×10^{-4}	0.595	0.507	5	0.978	0.963	1
2.5×10^{-3}	5.0×10^{-3}	0.630	0.510	6	0.979	0.963	2
2.5×10^{-3}	5.0×10^{-2}	0.623	0.509	6	0.974	0.959	3

Table 11: Training hyperparameter search on GPT-2 base with LoRA ($r_q = r_v = 8$)

Rank	Target	Trainable	SST Dataset (Batch = 256)			IMDB Dataset (Batch = 32)		
			Train Acc	Valid Acc	Epoch	Train Acc	Valid Acc	Epoch
2	K	40K (0.033%)	0.559	0.494	3	0.999	0.976	10
2	Q	40K (0.033%)	0.541	0.500	3	0.996	0.971	6
2	V	40K (0.033%)	0.629	0.514	8	0.978	0.971	2
2	O	40K (0.033%)	0.583	0.519	5	0.990	0.967	5
2	Q, V	78K (0.062%)	0.634	0.509	9	0.996	0.976	6
2	K, Q, V, O	151K (0.12%)	0.653	0.522	13	0.995	0.984	8
4	K	78K (0.062%)	0.608	0.495	9	0.992	0.967	4
4	Q	78K (0.062%)	0.599	0.483	9	0.971	0.967	2
4	V	78K (0.062%)	0.600	0.510	8	0.989	0.980	4
4	O	78K (0.062%)	0.832	0.518	10	0.996	0.976	11
4	Q, V	151K (0.12%)	0.640	0.529	8	0.998	0.976	6
4	K, Q, V, O	299K (0.24%)	0.578	0.508	12	0.996	0.976	9
8	K	151K (0.12%)	0.593	0.488	7	0.984	0.980	3
8	Q	151K (0.12%)	0.599	0.502	8	0.996	0.980	8
8	V	151K (0.12%)	0.579	0.526	6	0.988	0.984	4
8	O	151K (0.12%)	0.622	0.510	9	0.998	0.976	12
8	Q, V	299K (0.24%)	0.618	0.507	11	0.996	0.980	7
8	K, Q, V, O	594K (0.47%)	0.544	0.494	22	0.991	0.976	11

Table 12: LoRA Hyperparameter experiments with 25 epochs, GPT-2 Base. Base learning rate is adaptively adjusted with rank. Base lr: 2: ($5e-3$, $7.5e-4$), 4: ($3.2e-3$, $5e-4$), 8: ($2e-3$, $2.5e-4$). Note: For Q, V, learning rate is scaled by 0.5. For K, Q, V, O, learning rate is scaled by 0.25.

Setting	Full	PEFT
Seed	42	42
LR	$\{1, 2, 3\}e-5$	$\{1, 1.5, 2\}e-4$
Epochs / BS	$\{8, 10, 12\}/8$	$\{10, 12\}/4$
(p, T)	$(0.9, \{1.1, 1.2\})$	$(\{0.95, 0.97\}, \{0.9, 1.0\})$
Top- k	50	50
WD / Workers	0.05 / 4	0.05 / 4

Table 13: Hyperparameter Search Space for Sonnet Generation

Method	Key Config	LR	Epoch / BS	chrF
Adapter + LoRA	Adapt dim 64, rank 8, $\alpha = 32$, QV	1.5e-4	12 / 4	41.9648
Prefix Tuning	$n = 10$, bottleneck 64, interm. 512	1.5e-4	12 / 4	41.3166
LoRA + Prefix	Rank 8, $\alpha = 32$, QV + $n = 10$	1.5e-4	12 / 4	41.0642
Adapter + LoRA + Prefix	Adapt dim 64, rank 8, QV + $n = 10$	1.5e-4	12 / 4	40.4294

Table 14: Representative PEFT results for sonnet generation